

## 第 12 章 高级专题

### 学习目标

- ☑ 了解 DFA、NFA 和正规表达式的概念
- ☑ 理解 DAWG 与后缀自动机的概念及常见用法
- ☑ 掌握树的点分治算法
- ☑ 理解树的欧拉路径以及 LCA 和 RMQ 的关系
- ☑ 理解树的轻重路径剖分和 Link-Cut 树
- ☑ 了解可持久化数据结构的原理和典型实现
- ☑ 理解多边形布尔运算的原理和应用（如多边形偏移）
- ☑ 了解缓冲数据结构和分层数据结构的思想
- ☑ 掌握启发式合并、块链表、懒标记等数据结构设计思想和工具
- ☑ 学会用非完美算法求解问题
- ☑ 初步了解 OOP
- ☑ 初步了解函数式编程与 LISP
- ☑ 初步了解交互式题目

本章是全书最后一章，也是难度最高的一章。在第 11 章的末尾我们已经提到，如要顺利阅读本章内容，除了需要熟练掌握前 11 章的内容外，还需要熟悉本书的姊妹篇——《算法竞赛入门经典——训练指南》（以下简称《训练指南》）的大部分内容。

### 12.1 知识点选讲

#### 12.1.1 自动机

**有限自动机。**一个 DFA (Deterministic Finite Automaton, 确定有限状态自动机) 可以用一个 5 元组  $(Q, \Sigma, \delta, q_0, F)$  表示，其中  $Q$  为状态集， $\Sigma$  为字母表， $\delta$  为转移函数， $q_0$  为起始状态， $F$  为终态集。

这个 DFA 代表一个字符串集合。如何判断一个字符串是否属于这个集合（称为“被这个 DFA 接受”）呢？方法是边读边进行状态转移。一开始时，自动机在起始状态  $q_0$ ，每读入一个字符  $c$  后，状态转移到  $\delta(q, c)$ ，其中  $q$  为当前状态。当整个字符串读完之后，当且仅当  $q$  在终态集  $F$  中时，DFA 接受这个字符串。如图 12-1 所示， $Q = \{S_1, S_2\}$ ， $\Sigma = \{0, 1\}$ ， $q_0 = S_1$ ， $F = \{S_1\}$  (用双圈表示)，状态转移函数用转移弧来表示 (如  $S_1$  上面标有 1 的弧表示  $\delta(S_1, 1) = S_1$ )：

不难发现，上面的 DFA 接受的字符串集合是：0 的个数为偶数的 01 串。

NFA (Nondeterministic Finite Automata, 非确定自动机) 和 DFA 差不多，唯一的区别

是状态转移函数返回的是一个集合（可能是空集！）而不是一个状态，实际转移到集合中的任何一个状态（所以是“非确定性”）。如图 12-2 所示，从  $p$  出发有两条标记为 1 的弧，即  $\delta(p,1)=\{p,q\}$ 。

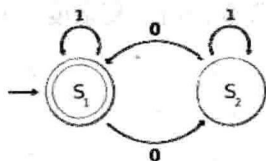


图 12-1 DFA 示例

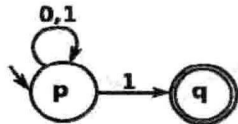
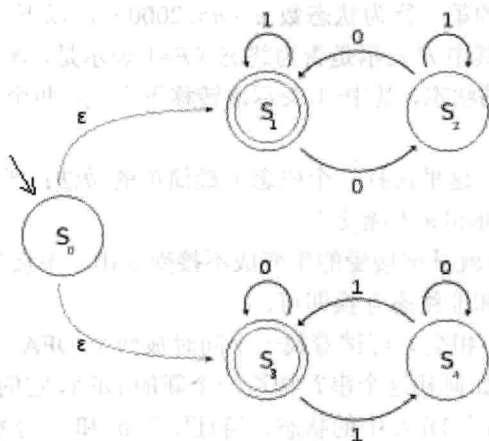


图 12-2 NFA 示例

不难发现，上面的 NFA 接受的字符串集合是：以 1 结尾的 01 串。NFA 有一个变种，即  $\epsilon$ -NFA，它和 NFA 的唯一区别是：可以有标记为  $\epsilon$  的转移弧，表示不需要输入任何一个字符就可以完成转移。下面是一个例子，如图 12-3 所示，接收的字符串集合是：0 的个数为偶数或者 1 的个数为偶数。

图 12-3  $\epsilon$ -NFA 示例

仔细观察这个自动机会发现：它实际上是两个 DFA 的并。上面的 DFA（起始状态为  $S_1$ ）表示“0 的个数为偶数”，下面的 DFA（起始状态为  $S_3$ ）表示“1 的个数为偶数”。

给定一个  $\epsilon$ -NFA，如何判断一个字符串是否被它接受？为方便起见，一般会先把  $\epsilon$ -NFA 转化为等价的 NFA，方法是先求出每个状态的所谓“ $\epsilon$ -闭包”，即只允许经过  $\epsilon$ -转移弧时可以到达的状态集（例如图 12-3 中  $S_0$  的闭包为  $\{S_0, S_1, S_3\}$ ），然后把每个状态转移  $\delta(q,c)=S$  改成  $\delta(q,c)=S'$ ，其中  $S'$  等于  $S$  中所有状态的  $\epsilon$ -闭包的并集。这样，就去掉了所有的  $\epsilon$ -转移。不过需要注意的是，这个 NFA 的起始状态有多个，它等于原  $\epsilon$ -NFA 的起始状态的  $\epsilon$ -闭包。例如，对于图 12-3，得到的 NFA 如图 12-4 所示，其中起始状态集为  $\{S_0, S_1, S_3\}$ 。注意，这个 NFA 包含了 3 个互不相干的部分。

假定字符串为 010，可以用递推的方法求出输入每个字符之后的状态集。

起始状态集： $\{S_0, S_1, S_3\}$ 。

输入字符 0 之后： $\{S_2, S_3\}$ 。

输入字符 1 之后:  $\{S_2, S_4\}$ 。

输入字符 0 之后:  $\{S_1, S_4\}$ 。

因为状态集中包含终态  $S_1$ , 串 010 被接受。不难把上述过程推广到一般情况, 如果 NFA 的状态个数为  $m$ , 字符串长度为  $n$ , 则判断该串是否被接受的时间复杂度为  $O(mn)$ 。

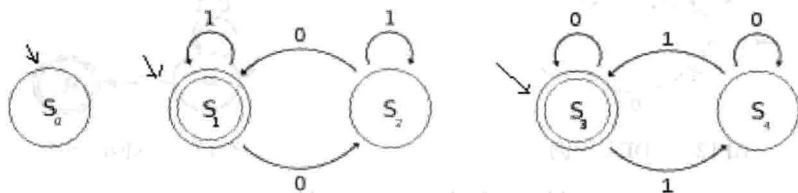


图 12-4 由图 12-3 得到的 NFA

### 例题 12-1 语言的历史 (History of Languages, ACM/ICPC Hangzhou 2008, UVa1671)

输入两个 DFA, 判断是否等价。第一行为字母表的大小  $T$  ( $2 \leq T \leq 26$ ), 然后是两个 DFA 的描述。每个 DFA 的第一行为状态数  $n$  ( $n \leq 2000$ ), 以下  $n$  行每行描述一个状态, 格式为  $F, X_0, X_1, \dots, X_{T-1}$ , 其中  $F$  表示是否为终态 ( $F=1$  表示是, 0 表示否)。 $-1 \leq X_i < N$ , 表示该状态读入  $i$  后转移到的状态, 其中  $-1$  表示该转移不存在。两个 DFA 的起始状态均为 0。

#### 【分析】

本题的做法不止一种, 这里选择一个概念上最简单的做法: 把“a 和 b 等价”转化为“a 的补和 b 不相交, 且 b 的补和 a 不相交”。

如何求 DFA 的补? 也就是把接受的串变成不接受的串, 不接受的串变成接受的串。由此可以想到, 只需把终态和非终态互换即可。

如何判断两个 DFA 不相交? 可试着找一个同时被两个 DFA 接受的串, 如果找不到, 则说明两个 DFA 不相交。如何找这个串? 构造一个新的 DFA, 它的每个状态都可以写成  $(q_1, q_2)$ , 其中  $q_1$  和  $q_2$  分别是两个 DFA 中的状态, 当且仅当  $q_1$  和  $q_2$  分别是两个 DFA 的终态时,  $(q_1, q_2)$  是新 DFA 的终态。这样, 问题就转化为了: 找一个被新 DFA 接受的串。这只需要用经典的图遍历 (DFS 或 BFS) 即可, 时间复杂度为  $O(n^2)$ 。

本题还有一个细节, 即对于“该转移不存在”的处理。虽然可以直接处理, 但更经典的方法是加一个“所有转移都指向自己”的“孤岛状态”, 把所有不存在的转移都改成转移到孤岛。这样一来, 所有转移都是存在的, 程序比较好写。

### 例题 12-2 不相交的正规表达式 (Disjoint Regular Expressions, ACM/ICPC NEERC 2012, UVa1672)

输入两个正规表达式, 判断二者是否不相交 (即不存在一个串同时满足两个正规表达式)。本题的正规表达式比较简单, 只包含以下几种情况。

- 单个小写字母  $c$ 。
- 或:  $(P|Q)$ 。如果字符串  $s$  满足  $P$  或者满足  $Q$ , 则  $s$  满足  $(P|Q)$ 。
- 连接:  $(PQ)$ 。如果字符串  $s_1$  满足  $P$ ,  $s_2$  满足  $Q$ , 则  $s_1s_2$  满足  $(PQ)$ 。
- 克莱因闭包:  $(P^*)$ 。如果字符串  $s$  可以写成 0 个或多个字符串  $s_i$  的连接  $s_1s_2\dots$ , 且每个串都满足  $P$ , 则  $s$  满足  $(P^*)$ 。注意, 空串也满足  $(P^*)$ 。

另外，多余的括号可以省略，克莱因闭包的优先级最高，其次是连接，最后是或。例如， $abc^*|de$  表示  $(ab(c^*))|(de)$ 。

输入的两个正规表达式 P 和 D 均不超过 100 个字符。如果 P 和 D 不是不相交的，应输出一个字符串，同时满足 P 和 D。例如， $a(ab)^*b$  和  $a(a|b)^*ab$  是不相交的，但  $a(ab)^*a$  和  $a(a|b)^*ba$  不是不相交的，因为  $aaba$  同时满足二者。

### 【分析】

正规表达式 (regular expression, 也译为正则表达式) 是进行文本处理的有力工具。对它的完整讨论超出了本书的范围，但是本题的解法仍然是支持更复杂的正规表达式语法的基础。

例 12-2 中用到的是 DFA，但是本题似乎很难直接从正规表达式构造 DFA，因为 DFA 有一个很强的限制：每个转移都是确定性的。如果放宽这一限制，是否能构造出 NFA 甚至  $\epsilon$ -NFA 呢？

幸运的是， $\epsilon$ -NFA 并不难构造<sup>①</sup>。图 12-5 中分别是单字符的自动机、 $(A|B)$  的自动机、 $(AB)$  的自动机和  $(A^*)$  的自动机。

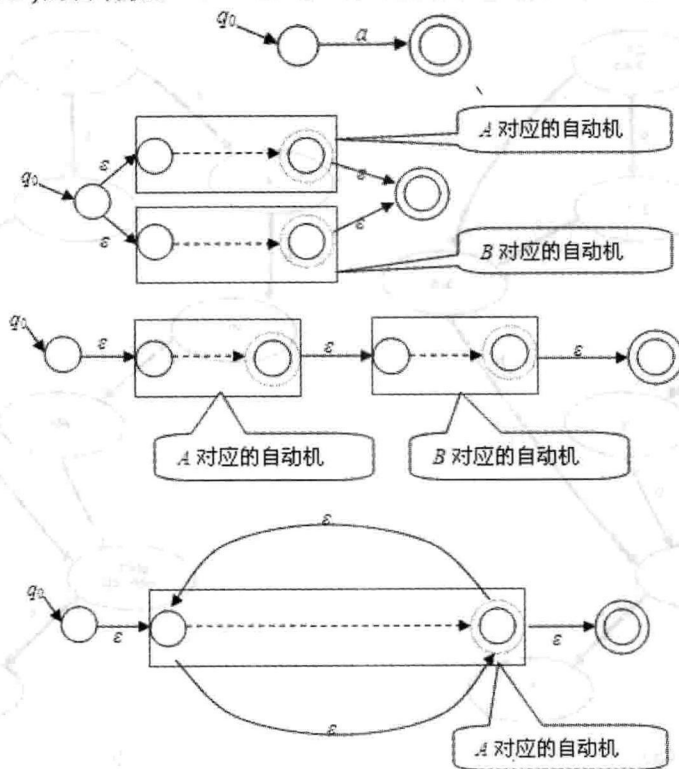


图 12-5 单字符、 $(A|B)$ 、 $(AB)$  和  $(A^*)$  自动机

从上面的自动机可以清楚地看到构造原理，不过状态有点多。

□  $(A|B)$  的自动机中可以把 A、B 和整个自动机的起点合并成一个点，把 A、B 和整

<sup>①</sup> 下面的方法称为 TCA (Thompson's Construction Algorithm)。

个自动机的终点也合并。

□ (AB)自动机中可以把整个自动机的起点和 A 的起点合并，A 的终点和 B 的起点合并，B 的终点和整个自动机的终点合并。

□ (A\*)自动机中可以把 A 的起点和终点合并。

现在已经拥有两个  $\varepsilon$ -NFA 了。为了方便起见，先把得到的两个  $\varepsilon$ -NFA 转化为 NFA。接下来就可以采用和上一题相同的思路，用 BFS 寻找一个同时被两个自动机接受的非空串了。注意这个串必须非空，所以要用三元组  $(q_1, q_2, b)$  来描述状态，表示两个自动机分别处于状态  $q_1$  和  $q_2$ ， $b=0$  表示没有进行过非  $\varepsilon$  转移， $b=1$  表示进行过。

DAWG。有一种特殊的自动机 DAWG (Directed Acyclic Word Graph)<sup>①</sup>，简记为  $D_w$ ，可以接受一个字符串  $w$  的所有子串，而且状态只有  $O(n)$  个，其中  $n$  是  $w$  的长度。

听上去很神奇吧？理解 DAWG 的关键是 end-set。一个单词的 end-set 是它在  $w$  中出现位置（从 1 开始编号）的右端点集合。例如，对于  $w=abcbc$ ， $\text{end-set}_w(bc)=\text{end-set}_w(c)=\{3, 5\}$ 。在 DAWG 中，end-set 相同的子串属于同一个状态。如图 12-6 所示是  $w=abcbc$  的 DAWG 的两种画法，其中图 12-6 (a) 中的结点里写着 end-set，图 12-6 (b) 的结点里写着子串集合本身。

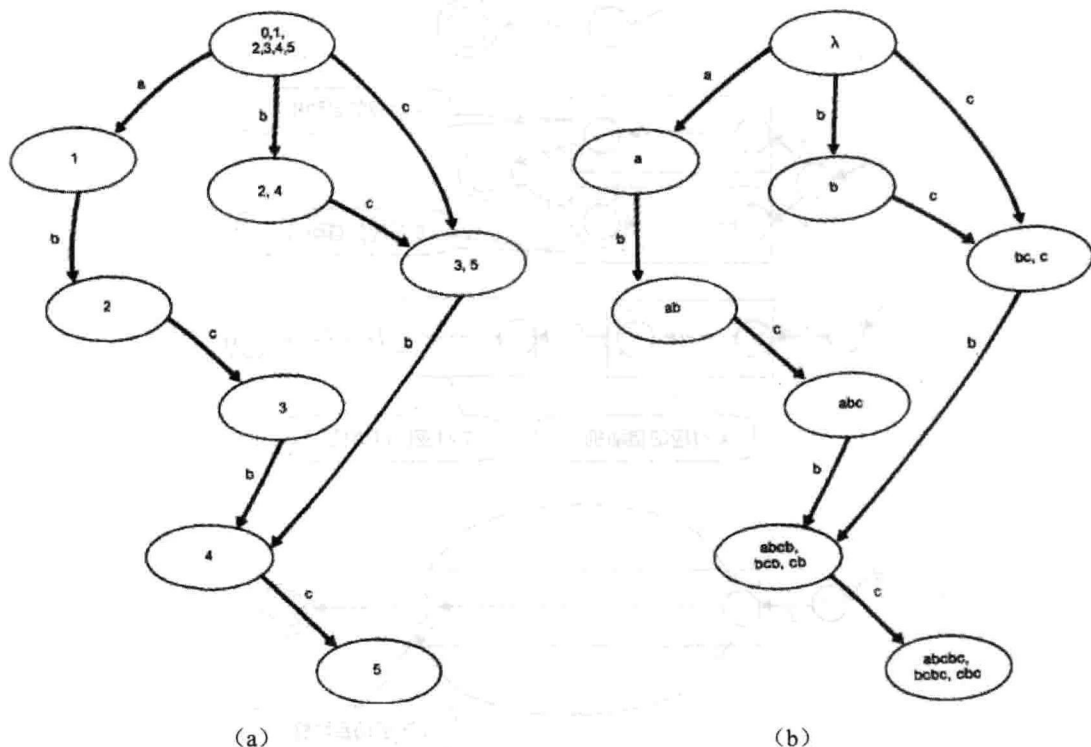


图 12-6  $w=abcbc$  的 DAWG

对于任意结点  $S$ ，从根结点到  $S$  的路径与  $S$  中的字符串是一一对应的，并且所有路径上的各个字母连接起来就是  $S$  中对应的那个字符串。例如，end-set 为  $\{4\}$  的结点中有 3 个串  $abcb$ ，

<sup>①</sup> 见 A. Blumer 等人于 1985 年写的经典论文：《The Smallest Automaton Recognizing the Subwords of a Text》。

bcb, cb, 从根结点到该结点的 3 条路径分别为  $a \rightarrow b \rightarrow c \rightarrow b$ 、 $b \rightarrow c \rightarrow b$  和  $c \rightarrow b$ 。另外, 每个状态中都有一个最长串, 其他的都是它的后缀, 并且长度连续。

任意两个结点的 end-set 要么不相交 (没有公共元素), 要么其中一个为另一个的子集, 因此可以得到一个树状结构  $T(w)$ , 如图 12-7 所示。

图 12-7 (a) 中的虚线是 DAWG 中的边, 实线是  $T(w)$  的边。这棵树其实是  $w$  的逆序串的后缀树, 如图 12-7 (b) 所示。 $T(w)$  最重要的性质就是: 对于任意一个结点  $S$ , 假设它的最长子串为  $x$ , 则  $x$  的所有后缀就是  $S$  及其所有祖先结点中的字符串集合。例如, 字符串  $abc$  是结点  $\{abc\}$  的最长串, 它和它的祖先  $\{bc, c\}$  与  $\{\text{空串}\}$  就是  $abc$  的后缀集。

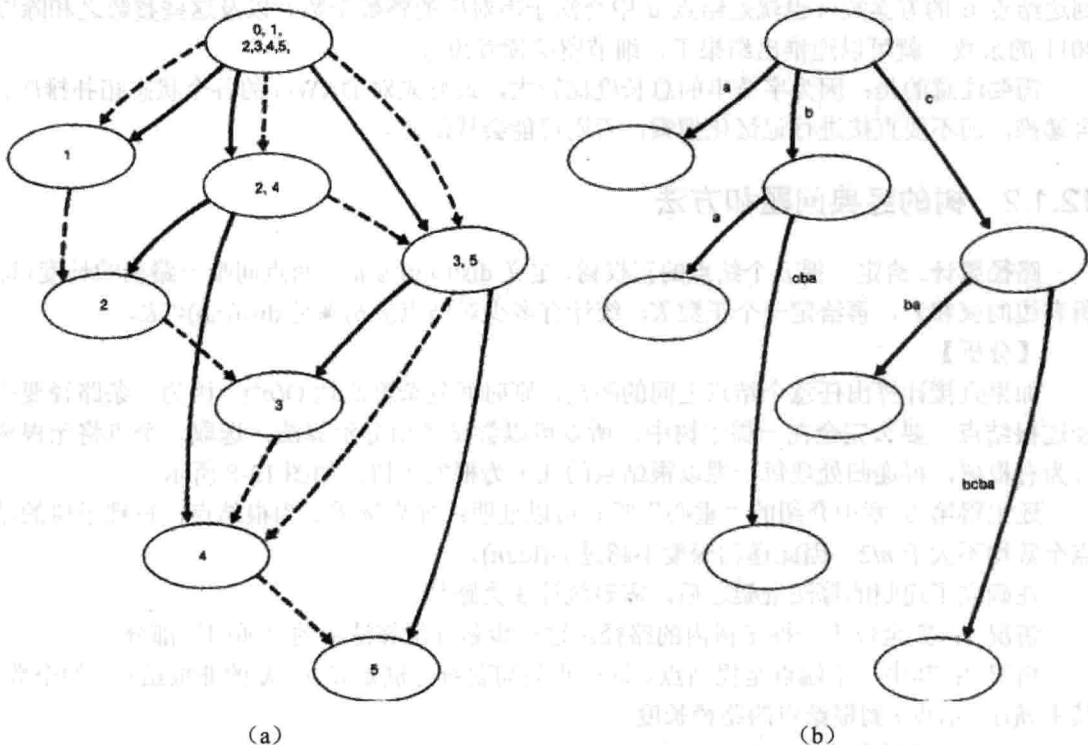


图 12-7 树状结构  $T(w)$

DAWG 可以在线性时间内在线构造, 即每次在字符串末尾添加一个字符后, 只需  $O(1)$  时间就可以更新 DAWG。不过对该构造算法的具体讨论超出了本书的范围, 强烈建议读者在网上搜索相关资料, 学会了 DAWG 的构造算法以后再看下面的例题。另外需要特别指出的是, end-set 中包含元素  $n$  的状态对应  $w$  的后缀。如果只把那些状态设为接受态, 则可以得到一个后缀自动机 (suffix automaton, SAM)。一般来说, 介绍后缀自动机的文献中讲的“后缀自动机的构造算法”实际上就是 DAWG 的构造算法。

### 例题 12-3 数字子串的和 (str2int, ACM/ICPC Tianjin 2012, UVa1673)

输入  $n$  ( $n \leq 10000$ ) 个数字串 (即由 0~9 组成的字符串), 把所有数字串的所有连续子串提取出来转化为整数, 然后去掉重复整数。例如, 两个数字串 101 和 123 可以得到 8 个整数: 1, 10, 101, 2, 3, 12, 23, 123。求这些整数之和除以 2011 的余数。所有数字串的长度之



和不超过  $10^5$ 。

### 【分析】

DAWG 在概念上很适合这道题目：每个状态里的字符串集合就是不同的子串集合。不过要想完整地解决本题，还有两个障碍。第一，本题的数字串有多个，而 DAWG 是针对单个字符串的；第二，因为数字 0 的存在，两个不同子串可能对应同一个整数。

第一个问题的解决方案在《训练指南》中已经介绍过了。设输入的数字串为  $w_1, w_2, \dots, w_n$ ，把它们拼成一个长串  $w = w_1\$w_2\$ \dots \$w_n$  后，构造  $w$  的 DAWG。第二个问题需要用递推来解决。从根结点开始走，规定不能走 \$ 边，且第一次不能走 0 边。设  $c(u)$  和  $s(u)$  分别表示到达结点  $u$  的方案数（也就是结点  $u$  中合法子串对应的整数个数）以及这些整数之和除以 2011 的余数，就可以递推出结果了，细节留给读者思考。

需要注意的是：因为字符串的总长度比较大，最好先对 DAWG 的各个状态拓扑排序，再递推，而不要直接进行记忆化搜索，否则可能会栈溢出。

## 12.1.2 树的经典问题和方法

**路径统计。**给定一棵  $n$  个结点的正权树，定义  $\text{dist}(u, v)$  为  $u, v$  两点间唯一路径的长度（即所有边的权和），再给定一个正数  $K$ ，统计有多少对结点  $(a, b)$  满足  $\text{dist}(a, b) \leq K$ 。

### 【分析】

如果直接计算出任意个结点之间的距离，则时间复杂度高达  $O(n^2)$ 。因为一条路径要么经过根结点，要么完全在一棵子树中，所以可以尝试使用分治算法：选取一个点将无根树转为有根树，再递归处理每一棵以根结点的儿子为根的子树，如图 12-8 所示。

还记得第 9 章中介绍的“重心”吗？可以证明：如果选重心为根结点，每棵子树的结点数均不大于  $n/2$ ，因此递归深度不超过  $O(\log n)$ 。

在确立了递归的算法框架之后，需要统计 3 类路径。

情况 1：完全位于一棵子树内的路径。这一步是分治算法中的“递归”部分。

情况 2：其中一个端点是根结点。这一步只需要统计满足  $d(i) \leq K$  的非根结点  $i$  的个数，其中  $d(i)$  表示点  $i$  到根结点的路径长度。

情况 3：经过根结点的路径。这种情况比较复杂，需要继续讨论。

记  $s(i)$  表示根结点的哪棵子树包含  $i$ ，那么要统计的就是：满足  $d(i) + d(j) \leq K$  且  $s(i)$  不等于  $s(j)$  的  $(i, j)$  个数，如图 12-9 所示。

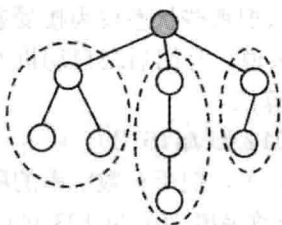


图 12-8 分治算法

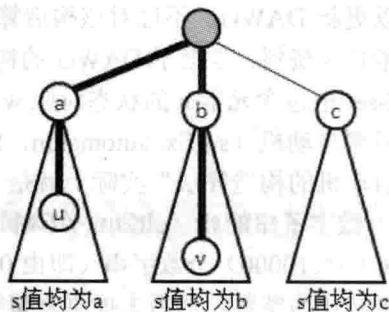


图 12-9 符合条件的  $s(i)$

由图 12-9 可看出, 任意两个  $s$  值不同的点之间都是一条经过根的路径, 可以使用补集转换。

设  $A$  为满足  $d(i)+d(j) \leq K$  的  $(i, j)$  个数,  $B$  为满足  $d(i)+d(j) \leq K$  且  $s(i)=s(j)$  的  $(i, j)$  个数, 则答案等于  $A-B$ 。如何计算  $A$  呢? 首先把所有  $d$  值排序, 然后进行一次线性扫描即可。  $B$  的计算方法也一样, 只不过是对于根的每个子结点分别处理, 把  $s$  值等于该子结点的所有  $d$  值排序, 然后线性扫描。根据主定理, 算法的总时间复杂度为  $O(n(\log n)^2)$ 。

上面介绍的是基于点的分治算法。实际上, 还有基于边和链的分治算法, 有兴趣的读者可以参考相关资料。

#### 例题 12-4 铁人比赛 (Ironman Race in Treeland, ACM/ICPC Kuala Lumpur 2008, UVa12161)

给定一棵  $n$  个结点的树, 每条边包含长度  $L$  和费用  $D$  ( $1 \leq D, L \leq 1000$ ) 两个权值。要求选择一条总费用不超过  $m$  的路径, 使得路径总长度尽量大。输入保证有解,  $1 \leq n \leq 30000$ ,  $1 \leq m \leq 10^8$ 。

##### 【分析】

沿用前面的分治算法框架, 关键问题就是如何计算经过树根的最优路径。首先用 DFS 求出子树内所有结点到根的路径长度和费用, 然后按照 DFS 序从小到大枚举这些结点。枚举到结点  $i$  时, 假设它到根的路径的费用为  $c(i)$ , 则需要在  $i$  之前的结点 (即已经枚举过的结点) 中找一个费用不超过  $D-c(i)$  的前提下, 到根结点距离最大的结点  $u$ 。

注意, 对于两个结点  $u$  和  $u'$ , 如果  $u$  到根的路径费用比  $u'$  大但路径长度比  $u'$  小, 则  $u$  一定不是最优解的端点, 可以删除。这样,  $i$  之前的结点可以组织成单调集合: 到根的路径长度和路径费用同时递增。如果把这个单调集合保存到 BST 中, 就可以在  $O(\log n)$  的时间找到“费用不超过给定值的前提下距离最大的结点”。这样, 在  $O(n \log n)$  时间内求出了“经过树根的最优路径”。根据主定理, 总时间复杂度为  $O(n(\log n)^2)$ 。

还有一种方法, 即求解子树时“顺便”把单调集合也构造出来。如果细节处理得当 (需要避开 BST), 还可以把计算“经过树根的最优路径”的时间复杂度降为  $O(n)$ , 细节留给读者思考。

**欧拉序列。**对 rooted 树  $T$  进行 DFS (深度优先遍历), 无论是递归还是回溯, 每次到达一个结点时都将编号记录下来, 可以得到一个长度为  $2N-1$  的序列, 称为树  $T$  的欧拉序列  $F$  (类似于欧拉回路)。

如图 12-10 所示, 结点 1 的深度为 0, 结点 2, 3, 4 的深度为 1, 结点 5, 6 的深度为 2, 因此欧拉序列  $F$  和深度序列  $B$  如表 12-1 所示。

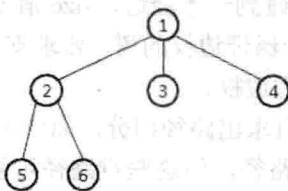


图 12-10 欧拉序列



表 12-1 欧拉序列 F 和深度序列 B

| 序号 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|----|---|---|---|---|---|---|---|---|---|----|----|
| F  | 1 | 2 | 5 | 2 | 6 | 2 | 1 | 3 | 1 | 4  | 1  |
| B  | 0 | 1 | 2 | 1 | 2 | 1 | 0 | 1 | 0 | 1  | 0  |

为了方便，把结点  $k$  在欧拉序列中第一次出现的序号记为  $\text{pos}(k)$ ，则图 12-10 中各个结点的  $\text{pos}$  值分别为 1, 2, 8, 10, 3, 5。欧拉序列中每个结点的第一次出现用灰色背景表示。

有了欧拉序列，LCA 问题可以在线性时间内转化为 RMQ 问题： $\text{LCA}(T, u, v) = \text{RMQ}(B, \text{pos}(u), \text{pos}(v))$ 。这里的 RMQ 返回值是下标而不是值本身。

这个等式不难理解：从  $u$  走到  $v$  的过程中一定会经过  $\text{LCA}(T, u, v)$ ，但不会经过  $\text{LCA}(T, u, v)$  的祖先。因此，从  $u$  走到  $v$  的过程中，深度最小的那个结点就是  $\text{LCA}(T, u, v)$ 。

用 DFS 计算欧拉序列的时间复杂度是  $O(N)$ ，且欧拉序列的长度为  $2N-1 = O(N)$ ，所以 LCA 问题可以在  $O(N)$  的时间内转化为等规模的 RMQ 问题。

**树的动态查询问题 I。** 给定一棵带边权的树，要求支持两种操作：修改某条边的权值和询问树中某两点间的距离。

首先把无根树变成有根树，则把一条边  $u-v$ （假定  $u$  是  $v$  的父结点）的权值增加  $d$  时，以  $v$  为根的整个子树的“到根结点的距离”同时增加  $d$ 。不难发现，一棵子树内的结点对应欧拉序列中的一段连续序列，因此如果用  $\text{dist}[i]$  表示欧拉序列中第  $i$  个结点到根的距离，则修改操作就是  $\text{dist}$  数组上的“区间增量”，而查询时的距离  $\text{dist}(u, v)$  等于  $\text{dist}(u) + \text{dist}(v) - 2\text{dist}(w)$ ，其中  $w = \text{LCA}(u, v)$ 。这样，只需用一个支持快速区间增量和单点查询的数据结构（例如 Fenwick 树或者线段树）来维护  $\text{dist}$  数组，就可以在  $O(\log n)$  时间内支持两个操作。

**轻重路径剖分。** 给定一棵有根树，对于每个非叶结点  $u$ ，设  $u$  的子树中结点数最多的子树的树根为  $v$ ，则标记  $(u, v)$  为重边，从  $u$  出发往下的其他边均为轻边，如图 12-11 所示（结点中的数字代表结点的  $\text{size}$  值，即以该结点为根的子树的结点数）。

根据上面的定义，只需一次 DFS 就能把一棵有根树分解成若干重路径（重边组成的路径）和若干轻边。有些资料也把重路径称为树链，因此轻重路径剖分也称树链剖分。

路径剖分中最重要的定理如下：若  $v$  是  $u$  的子结点， $(u, v)$  是轻边，则  $\text{size}(v) < \text{size}(u)/2$ ，其中  $\text{size}(u)$  表示以  $u$  为根的子树中的结点总数。

证明并不复杂。由定义，所有非叶结点往下都有一条重边。假设  $\text{size}(v) \geq \text{size}(u)/2$ ，那么对于  $u$  向下的重边  $(u, w)$  来说， $\text{size}(w) \geq \text{size}(v) \geq \text{size}(u)/2$ ，因此  $\text{size}(u) \geq 1 + \text{size}(v) + \text{size}(w) \geq 1 + \text{size}(u)$ ，与假设矛盾。

由此可以得到如下的重要结论：对于任意非根结点  $u$ ，在  $u$  到根的路径上，轻边和重路径的条数均不超过  $\log_2 n$ ，因为每碰到一条轻边， $\text{size}$  值就会减半。

**树的动态查询问题 II。** 给定一棵带边权的树，要求支持两种操作：修改某条边的权值和询问树中某两点的唯一路径上最大边权。

首先把无根树变成有根树并且求出路径剖分。如图 12-12 所示，任意结点  $u$  到其祖先  $x$  的简单路径中包含一些轻边和重路径，但这些重路径可能并不是原树中的完整重路径，而只是一些“片段”，因此可以在轻边中直接保存边权，而用线段树维护重路径。

这样，两个操作都不难实现。

修改：轻边直接修改，重边需要在重路径对应的线段树中修改。

查询：设  $LCA(u, v) = p$ ，则只需求出  $u$  到其祖先  $p$  之间的最大边权  $\max w(u, p)$ ，再用类似的方法求出  $\max w(v, p)$ ，则答案为  $\max\{\max w(u, p), \max w(v, p)\}$ 。为了求出  $\max w(u, p)$ ，依次访问  $u$  到  $p$  之间的每条重路径和轻边即可。根据刚才的结论，轻边和重路径的条数均不超过  $\log_2 n$ 。这样，修改的时间复杂度为  $O(\log n)$ ，查询的时间复杂度为  $O(\log^2 n)$ 。虽然存在时间复杂度更低的方法<sup>①</sup>，但上述方法已经很实用了。

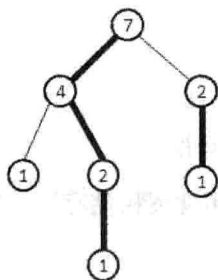


图 12-11 轻重路径剖分

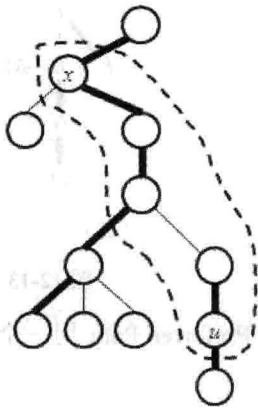


图 12-12 树的动态查询

**Link-Cut 树。**值得一提的是，轻重路径剖分有一个“动态版本”——Sleator 和 Tarjan 的 Link-Cut 树<sup>②</sup>。该数据结构解决的是所谓的动态树（Dynamic Tree）问题，即维护一个有根树组成的森林。支持以下 4 个操作。

- MAKE-TREE(): 创建一棵新树。
- CUT( $v$ ): 删除  $v$  到父亲的边，相当于把以  $v$  为根的子树独立出来。
- JOIN( $v, w$ ): 让  $v$  成为  $w$  的子结点。这里  $v$  必须是森林中一棵树的根，且  $w$  不在这棵树中。
- FIND-ROOT( $v$ ): 找出  $v$  所在树的根结点。

其中 CUT 和 JOIN 是两个最经典的操作，利用它们可以灵活地改变树的结构。“重路径”在 Link-Cut 树中称为 Preferred Path。每条 Preferred Path 用一棵辅助树表示（通常是伸展树<sup>③</sup>），而不同的辅助树之间通过父结点指针连在一起。

图 12-13 展示了 Link-Cut 树最重要的操作：Access 操作。Access( $u$ )的作用是把从根结点到  $u$  的路径变成重路径。为此，可能需要把一些其他的重边变成轻边以维持“每个非叶结点往下最多有一条重边”这一性质。图 12-13 (a) 执行 Access( $N$ )之后得到图 12-13 (b)，其中重边 A-B, H-J, I-K 都变成了轻边。另外，根结点和执行 Access 操作的结点必须是重路

<sup>①</sup> 出于时间和空间上的考虑，在竞赛中我们往往不是给每条重路径建一棵线段树，而是用一棵全局线段树保存所有树链，限于篇幅，这里不再详细介绍。

<sup>②</sup> 原始论文：<http://www.cs.cmu.edu/~sleator/papers/self-adjusting.pdf>。这里介绍的版本和原始论文有差异，在实践中更为常用。

<sup>③</sup> 原论文中不是使用的伸展树，因为 Link-Cut 树比伸展树更早发明。

径的两个端点，所以 N-O 也必须变成轻边。

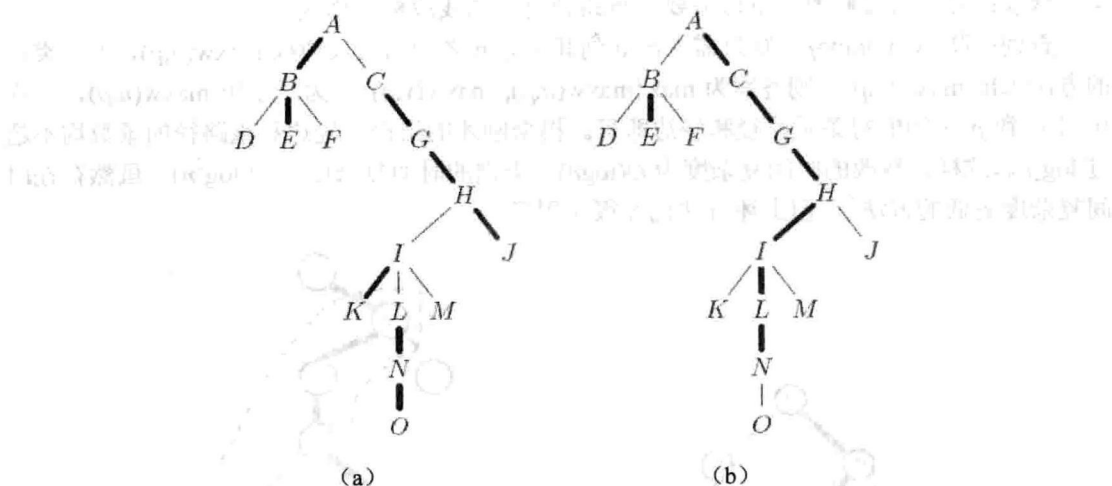


图 12-13 Link-Cut 树中 Access 操作

如果把每条 Preferred Path 用一个序列表示（实际上用伸展树储存），则上面两棵树如图 12-14 所示。

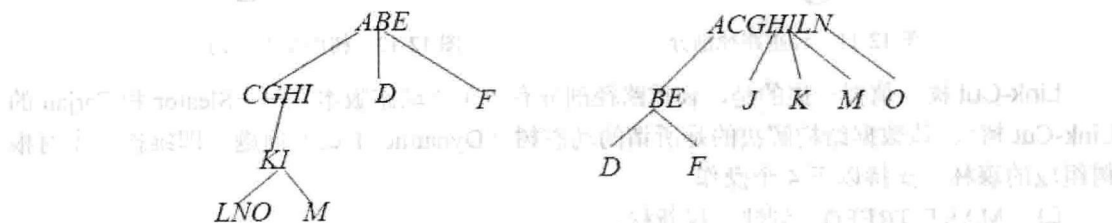


图 12-14 将 Preferred Path 用序列表示

对于 Link-Cut 树的完整讨论超出了本书的范围，建议读者熟练掌握它（包括时间复杂度和程序实现），之后再阅读下面的例题。

### 例题 12-5 快乐涂色 (Happy Painting, UVa11994)

$n$  个结点组成了若干棵有根树，树中的每条边都有一个特定的颜色。你的任务是执行  $m$  条操作，输出结果。操作一共有 3 种，如表 12-2 所示。

表 12-2 3 种操作

| 操 作     | 含 义                                                                   |
|---------|-----------------------------------------------------------------------|
| 1 x y c | 把 x 的父结点改成 y。如果 x=y 或者 x 是 y 的祖先，则忽略这条指令，否则删除 x 和它原先父结点之间的边，而新边的颜色为 c |
| 2 x y c | 把 x 和 y 的简单路径上的所有边涂成颜色 c。如果 x 和 y 之间没有路径，则忽略此指令                       |
| 3 x y   | 统计 x 和 y 的简单路径上的边数，以及这些边一共有多少种颜色                                      |

每组数据第一行为  $n$  和  $m$  ( $1 \leq n \leq 50000$ ,  $1 \leq m \leq 200000$ )，然后是每个结点的父结点编号和该结点与父结点之间的边的颜色（对于根结点，父结点编号为 0，且“与父结点之间

的边的颜色”无意义)。接下来是  $m$  条指令。对于所有指令,  $1 \leq x, y \leq n$ ; 对于类型 2 指令,  $1 \leq c \leq 30$ 。结点编号为  $1 \sim n$ , 颜色编号为  $1 \sim 30$ 。

对于每个类型 3 指令, 输出对应的结果。

### 【分析】

这是一个标准的动态树问题, 不过多了一个“统计颜色数”操作。注意到颜色只有 30 种, 可以用一个 32 位整数表示一个颜色集合。由于辅助树用伸展树保存, 可以在伸展树的每个结点中加一个信息  $c$ , 即以该结点为根的子树所对应的重路径“片段”所拥有的颜色集, 则操作 2 和 3 都对应于经典的伸展树的修改和查询操作。

### 例题 12-6 闪电的能量 (Lightning Energy Report, ACM/ICPC Jakarta 2010, UVa1674)

有  $n$  ( $n \leq 50000$ ) 座房子形成树状结构, 还有  $Q$  ( $Q \leq 10000$ ) 道闪电。每次闪电会打到两个房子  $a, b$ , 你需要把二者路径上的所有点 (包括  $a, b$ ) 的闪电值加上  $c$  ( $c \leq 100$ )。最后输出每个房子的总闪电值。

### 【分析】

出题者的标准解法是利用路径剖分: 每次最多更新  $2 \log n$  条重路径, 而每条重路径上的区间更新需要  $O(\log n)$  时间。

这样做也没有错, 但是有点小题大做。其实, 对于询问  $(a, b, c)$ , 可以首先算出  $d = \text{LCA}(a, b)$ , 然后执行  $\text{mark}[a] += c, \text{mark}[b] += c, \text{mark}[d] -= c$ 。如果  $d$  不是树根, 还要让  $d$  的父结点  $p$  的  $\text{mark}$  值减  $c$ 。原理是这样的:  $\text{mark}[u] = w$  的意思是  $u$  到根的路径上每个点的权都要加上  $w$ , 即结点  $i$  的闪电值等于根为  $i$  的子树的总  $\text{mark}$  值。如图 12-15 所示, 经过上述  $\text{mark}$  修改操作之后, 只有  $a$  到  $b$  路径上所有点的“子树总  $\text{mark}$  值”增加了  $c$ , 其他结点保持不变。

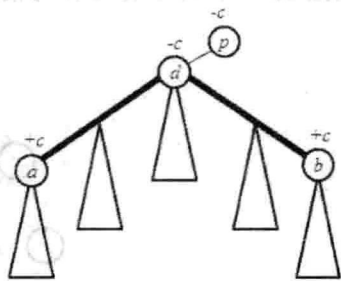


图 12-15 mark 修改操作后结果

最后用一次 DFS, 即可求出以每个结点为根的子树的总  $\text{mark}$  值。

## 12.1.3 可持久化数据结构

《训练指南》中介绍了一些基本的数据结构, 例如 BIT、线段树等, 也介绍了一些高级数据结构技巧, 例如嵌套数据结构和分块数据结构。但有一个重要的话题并未涉及, 那就是可持久化数据结构 (persistent data structures)。

之前学过的很多数据结构都是可变的, 所有修改操作都直接改变了数据结构本身。修改之后, 就无法得到修改之前的数据结构了。有时, 需要在修改数据结构之后得到的是该数据结构的一个新版本, 同时保留修改前的“老版本”。该如何实现呢?

基本思路是: 不许修改结点内的值; 必要时创建或者复制结点; 尽量复用存储空间。

如图 12-16 所示, 我们希望在链表的第 3 个结点后面新加一个白色结点, 只需要复制前 3 个结点即可。

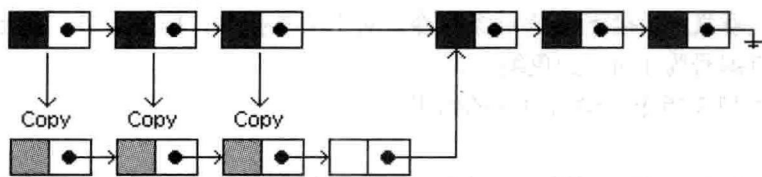


图 12-16 在链表结点中加入结点

虽然整个结构看上去比较奇怪，但是从两个链表各自的表头指针开始访问，沿途访问到的就是该链表自身的结点。

当然，这个例子并不是那么吸引人，因为平均情况下要复制一半的结点，不过这个方法可以用来实现一个可持久化的栈——在链表的头部进行入栈和出栈，不仅时间是  $O(1)$  的，附加空间也是  $O(1)$  的。

如果是一棵满的排序二叉树，没有插入和删除，只有修改，则不需要旋转操作，因此很容易用上述方法改造成可持久化的排序二叉树。修改单个结点时，只需把从根结点到修改结点的所有结点（只有  $O(\log n)$  个）复制一份并设置好链接关系，其他结点保持不变即可，如图 12-17 所示。把  $a$  作为根访问到的就是老树，把  $b$  作为根访问到的就是新树。

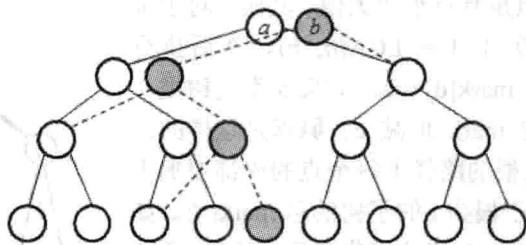


图 12-17 将满的排序二叉树改造成可持久化的排序二叉树

顺便一提：已经有一些编程语言中“自带”了可持久化数据结构，例如 Scala、Erlang 和 Clojure，有兴趣的读者可以参考这些语言的入门书籍，会对可持久化数据结构有一个更加清晰具体的认识。

**例题 12-7** 自带版本控制功能的 IDE (Version Controlled IDE, ACM/ICPC Hatyai 2012, UVa12538)

编写一个支持查询历史记录的编辑器，支持以下 3 种操作。

- 1 p s: 在位置  $p$  前插入字符串  $s$ 。
- 2 p c: 从位置  $p$  开始删除  $c$  个字符。
- 3 v p c: 打印第  $v$  个版本中从位置  $p$  开始的  $c$  个字符。

缓冲区一开始是空串，是版本 0，每次执行操作 1 或 2 之后版本号加 1。每个查询回答之后才能读到下一个查询。操作数  $n \leq 50000$ ，插入串总长不超过 1MB，输出总长保证不超过 200KB。

【分析】

本题要实现的数据结构就是一个典型的可持久化数据结构。在《训练指南》中曾经见过一道类似的例题，但是只需非持久化版本的题目：《排列变换》。在那道题目中，用到

了伸展树的 split 和 merge 操作，本题可以如法炮制。

**split 操作。**假定要把序列子树  $S$  分裂成  $L$  和  $R$  两部分，其中左边有 `left_size` 个结点。如果 `left_size` 小于  $S$  左子树的结点个数，则可以先递归调用 split 操作把  $S$  的左子树分裂为  $L$  和  $R'$ ，其中  $L$  的结点个数为 `left_size`，然后创建一个值  $w$  和新结点  $R$ ，左右子树分别为  $R'$  和  $S$  的右子树。不难发现， $L$  和  $R$  合起来正好是  $S$  的所有元素，并且  $L$  里有 `left_size` 个元素。`left_size` 比较大时也可以类似处理，如图 12-18 所示。

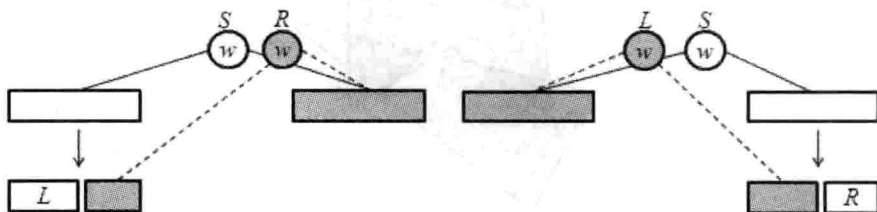


图 12-18 split 操作

**merge 操作。**假定要把两个序列  $a$  和  $b$  合并成一个序列  $S$ 。和 split 类似，也有两种方法合并，但两种方法都可以用，并不是上面的“二选一”。例如，图 12-19 (a) 就是先递归调用 merge 操作把  $a$  的右子树和  $b$  合并成  $R$ ，然后创建一个新结点  $S$ ，而图 12-19 (b) 则是相反。不管选哪种 merge 方式都有可能合并成一棵形态不好的树，所以随机合并。

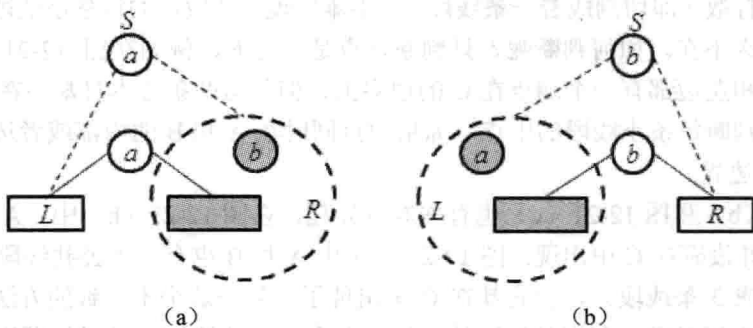


图 12-19 merge 操作

实际上，这就是一个可持久化 treap 的 split 和 merge 操作。对上述方法的理论分析超出了本书的范围，但可以告诉大家的是，它的实际效果非常好，并且程序易于实现，是可持久化数据结构的经典例子。

值得指出的是，如果可以使用 STL 扩展，那么用 rope 实现本题也是一个不错的选择。有兴趣的读者可以阅读维基百科<sup>①</sup>。

### 12.1.4 多边形的布尔运算

布尔运算是指把多边形看成一个点集，然后执行集合的布尔运算。最常见的布尔运算是交和并。虽然概念简单，但实际上多边形的布尔运算不是那么容易实现的。如果要高效

<sup>①</sup> [http://en.wikipedia.org/wiki/Rope\\_%28computer\\_science%29](http://en.wikipedia.org/wiki/Rope_%28computer_science%29)。



实现，更是难上加难。

**例题 12-8 多边形相交 (Polygon Intersections, ACM/ICPC World Finals 1998, UVa805)**

输入两个简单多边形，求二者相交的区域（如图 12-20 的深色区域所示）。如果有多个区域，应分别输出。共线的相邻边应合并（细节请参考原题）。

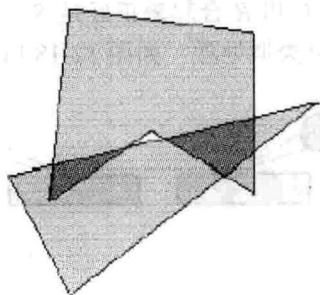


图 12-20 多边形相交

**【分析】**

为了叙述方便，设输入的多边形为  $A$ （用细线表示）和  $B$ （用粗线表示），答案为  $C$ （图中未画出），如图 12-21 所示。输入的是简单多边形，所以  $C$  是不会出现洞的，但是可能会不连通。算法大概是这样的：首先对于每条线段求出它和其他线段的交点，然后在交点处把线段打散（即切割成若干条线段）。不难发现，打散后的每条小线段要么完全在  $C$  的边界上，要么不在。如何判断呢？只判断端点是不行的，例如在图 12-21 (a) 中，细线正方形的上边和左边都有一个端点在  $C$  的边界上，但是这两条边本身却不在  $C$  的边界上。正确的做法是判断每条小线段的中点。如果中点同时在  $A$  和  $B$  的内部或者边界上，则这条小线段是  $C$  的边界。

图 12-21 (b) 和图 12-21 (c) 也有些难以处理。在图 12-21 (b) 中， $A$  和  $B$  有一条公共线段，但是并没有在  $C$  中出现；图 12-21 (c) 中  $A$  和  $B$  也有一条公共线段（注意  $A$  的右边界已被打断成 3 条线段），但它却在  $C$  里出现了。解决这个不一致的方法有多种，这里只介绍笔者认为相对常见和容易编写的一种：把多边形的边按照逆时针顺序定向，然后去掉重复的有向线段，如图 12-22 所示。

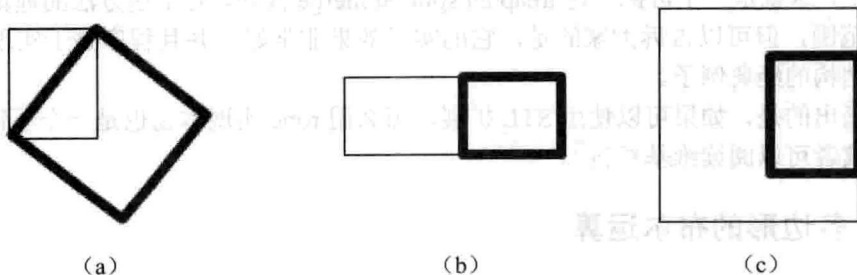


图 12-21 多边形相交问题分析

经过上述处理之后，得到了若干有向线段。只要把它们拼起来，然后把退化的多边形（折线）删除，只保留多边形区域，就得到了最终的答案。例如，图 12-22 (b) 拼起来以

后得到了一个只有两个点的“多边形”，输入退化情况，应删除。

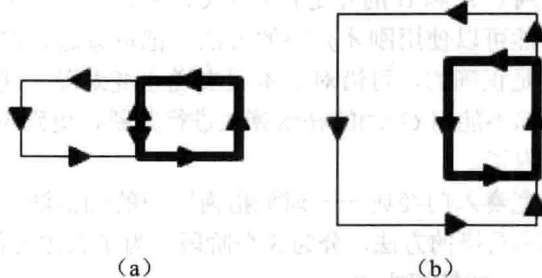


图 12-22 解决不一致问题的方法

### 例题 12-9 王国的重新合并 (Kingdom Reunion, ACM/ICPC NEERC 2012, UVa1675)

输入 3 个国家 Aastria、Abstria 和 Aabstria 的边界，判断 Aastria、Abstria 是否可以恰好不重叠地合并成 Aabstria。输入可能有误，即 3 个边界都可能不是多边形。输出有 6 种情况。

情况 1: 如果 Aastria 的边界不是合法多边形，输出 Aastria is not a polygon。

情况 2: 如果 Abstria 的边界不是合法多边形，输出 Abstria is not a polygon。

情况 3: 如果 Aabstria 的边界不是合法多边形，输出 Aabstria is not a polygon。

情况 4: 如果 Aastria 和 Abstria 相交，输出 Aastria and Abstria intersect。

情况 5: 如果 Aastria 和 Abstria 的合并不是 Aabstria，输出 The union of Aastria and Abstria is not equal to Aabstria。

情况 6: 输出 OK。

图 12-23 中 4 幅图分别对应情况 6、情况 1、情况 4、情况 5。输入中每个边界上的点数都不超过 10000。

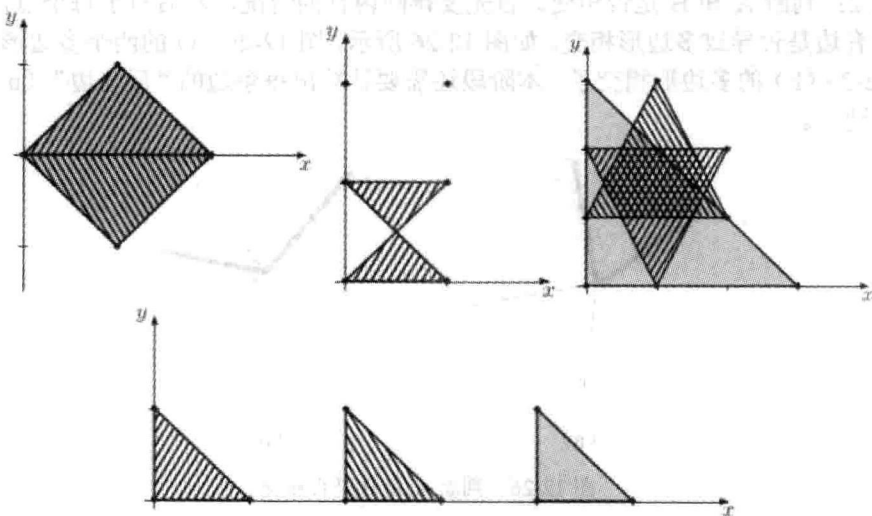


图 12-23 情况 6、情况 1、情况 4 和情况 5

#### 【分析】

本题的数据范围很大，但在优化之前要先思考一下：不考虑时间复杂度的情况下如何

求并。一般情况下，两个多边形 A 和 B 的“并”可能是一个“有洞多边形”，如图 12-24 所示。不过本题只需要判断 A 和 B 的并是否等于 C，所以可以不考虑这种情况。

不难发现，此处仍然可以使用刚才介绍的方法：把每条边定向，打断线段并判重，然后逐一判断。这个方法是正确的，可惜对于本题来说速度太慢，就连“判断多边形相交”和“打断线段”这一步都不能用  $O(n^2)$  的朴素算法进行判断，更别说判断每条（打断后的）线段是否在两个多边形内了。

解决方法是《算法竞赛入门经典——训练指南》中的扫描法。具体写法有很多种，这里只介绍一种相对不容易写错的方法，分为 3 个阶段。为了叙述方便，设 Aastria 和 Abstria 的轮廓为 A 和 B，Aabstria 的轮廓为 C。

阶段 1：用扫描法判断 A、B、C 是否为合法多边形。这一步看似简单，其实有陷阱。在扫描法中，新增或者删除线段时会判断相邻线段是否相交。这个“相交”一般会理解成“只要有公共点就算相交”，而不一定是规范相交。但是在本阶段中，如果这样写就错了（因为这两条线段可能恰好是同一个顶点出发的两条边）。另一方面，也不能把这里的“相交”理解成“规范相交”，因为图 12-25 中所示就不是规范相交，但它也不是一个合法多边形，应当被检测出来。阶段 1 的另一个作用是用所有顶点去打断每条边，具体细节留给读者思考。

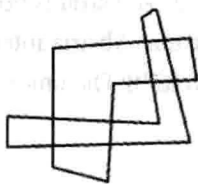


图 12-24 两个多边形的并

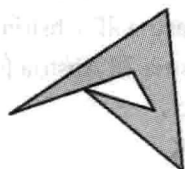


图 12-25 非规范相交

阶段 2：判断 A 和 B 是否相交。首先要排除内含的情况，然后对于每个点，判断从它出发的所有边是否导致多边形相交。如图 12-26 所示，图 12-26 (a) 的两个多边形没有相交，但是图 12-26 (b) 的多边形相交了。本阶段还需要计算出每条边的“反向边”（ $u \rightarrow v$  和  $v \rightarrow u$  互为反向边）。

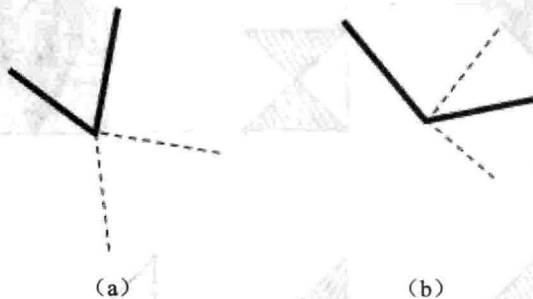


图 12-26 判断 A 和 B 是否相交

接下来就可以忽略同一个顶点出发的边了。再扫描一次，和阶段 1 一样判断线段相交。但是这次不需要打断线段，而且每到一个事件点时要把与它关联的所有相邻边一次性加到扫描线上，就不会认为这些边相交了。

阶段3: 判断A和B是否覆盖了C。以A为例, 首先枚举A的每条边 $u \rightarrow v$ , 看看C是否也有一条从 $u$ 出发的边。如果C中没有从 $u$ 出发的边, 则B中必须有边 $v \rightarrow u$ , 这样才能和A中的 $u \rightarrow v$ 相互“抵消”, 让C的边界中不必出现这条边。类似地, 如果C有一条完全相同的边 $u \rightarrow v$ , 则B中不能有边 $v \rightarrow u$ 。因为之前已经算过了反向边, 所以对于每个顶点 $u$ , 只需常数时间内就可以完成上述判断。

#### 例题 12-10 清洁机器人 (The Cleaning Robot, Rujia Liu's Present 4, UVa12314)

有一个半径为 $r$ 的圆形清洁机器人和一个 $n$  ( $n \leq 100$ ) 边形障碍。需要把机器人放到某个地方, 使得它无法移动到无穷远处, 要求能清洁到的区域面积尽量大。如图 12-27 所示, 图 12-27 (a) 的阴影部分就是能清洁到的区域, 而图 12-27 (b) 中有两个选择, 其中右边那个区域更大。

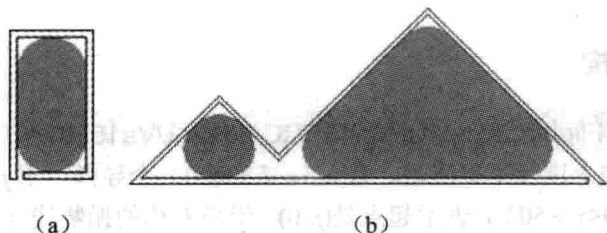


图 12-27 “清洁机器人”问题示意图

#### 【分析】

首先看看机器人的圆心可能在哪些位置。根据题意, 圆心不可能在多边形内部, 到多边形的距离也不能小于 $r$ , 所以可以设计一个“膨胀”操作<sup>①</sup>, 计算出圆心禁止出现的区域, 它实际上等于若干个矩形、若干个圆以及原多边形的并, 如图 12-28 所示。

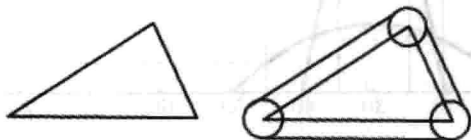


图 12-28 圆心禁止出现的区域

图 12-28 看上去很规则: 每条边外扩, 然后用每个顶点处的圆弧连接。但有时有些边会消失, 还是只能使用多边形并的算法, 如图 12-29 所示。

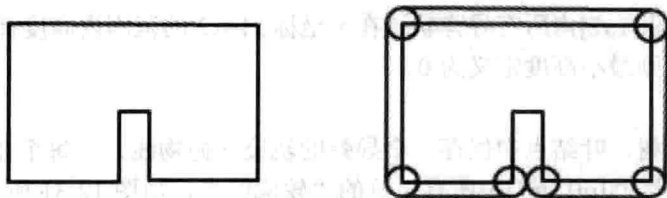


图 12-29 多边形并的算法

<sup>①</sup> 它的正式名称为多边形偏移 (offsetting)。